

Last modified on

Sports Data Soccer API - Automated Insights Feed (MA23)

Get real-time Insights and comments for a match

Overview

The Soccer **Automated Insights feed** provides 'in-game' real-time Insights for a specific match. This helps to provide additional context to the match which could include player/team milestones, match or seasonal records, and more. This content is automatically generated using live match and historical data to form written Insight comments.

If the Insight is associated with an event that has occurred in the game, then optional event data will also be provided, such as the **eventId** , associated player, team, minute and second for the action.

You can cross-reference the **eventId** attribute-value with the **eventId** attribute value that is delivered in the Match Events feed (SDAPI MA3 or SDDP MA3DP) for the same match (for the same unique fixture ID).

This data is explained in the **Feed Output** section of this guide.

Mapping an eventId:

- You can cross-reference an eventId with the data available for the same fixture UUID in the SDAPI **MA3** 'pull' feed or, for SDDP 'push' subscribers, the **MA3DP Match Events** SDDP websocket feed.
- To learn more about how to cross-reference the fixture UUID, eventId or other data points between the SDDP and SDAPI feeds, see **Mapping a fixture UUID and its eventIds across feeds to get diverse data coverage for a match** in the SDDP Integration Guide

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

Competition availability: Currently, this feed is available for a handful of competitions and languages including the English Premier League (EPL). Please check availability with your Account Manager on any new competition or languages.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY
Typically, we will send an Insight when something noteworthy happens in a game - for example, a goal, red card, milestone, or when the first/second half ends - but we may also send Insights in reaction to other events or observations about what's happening in a game	No more frequently than one URL poll every 30 seconds

Automated Insights (MA23) Guide

USAGE	FEED
GET automated insights by fixture UUID (path parameter method) and for the specified language/locale	/soccerdata/matchinsights/{outletAuthKey}/{fixtureUUID}&{queryParams}
GET automated insights by legacy Opta ID (fx query parameter method) and for the specified language/locale	/soccerdata/matchinsights/{outletAuthKey}?fx={optaId}&{queryParams}

If your request URL is as follows - that is, it does not contain a fixture UUID AND `_1c1` parameter and value - the feed returns an error:
`/soccerdata/matchinsights/{outletAuthKey}?_fmt={dataFormat}&_rt={mode}`

Query Parameters And Example Request Parameters

This feed also makes use of the [Global query parameters](#), and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/matchinsights` feed by a specific fixture UUID or legacy Opta fixture (match) ID.

NOTE: Return error code **10400** when there is no insight data available for a match, instead of returning basic match info data.

Legacy IDs: You can use the `fx` parameter to filter by legacy IDs - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example `GET` requests (rather than `cURL`, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the [main guide](#) and [API Overview and Authorisation guide](#).

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or `-` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you **MUST** use it in combination with the callback parameter (`_c1bk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>{fixtureUuid}</code> Fixture UUID	GET match insights for the specified fixture UUID (path parameter method) and locale . Pass the fixture UUID in the request URL directly as the <code>{assetId}</code> (asset/resource ID), and pass the fixture UUID to the <code>fx</code> parameter. GET <code>https://api.performfeeds.com/soccerdata/matchinsights/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}</code>
<code>fx</code>	GET match insights by the specified legacy Opta fixture/match ID (query parameter method) . Pass the Opta fixture/match ID to the <code>fx</code> parameter. For example: <code>fx=urn:perform:opta:fixture:987971</code> GET <code>https://api.performfeeds.com/soccerdata/matchinsights/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&fx=urn:perform:opta:fixture:{fixtureUuid}</code>
<code>state</code>	GET match insights by the specified state . The three states that can be passed to this query parameter are <code>live</code> , <code>post</code> , and <code>live,post</code> . Pass the state to the <code>state</code> parameter. For example: <code>state=live,post</code> GET <code>https://api.performfeeds.com/soccerdata/matchinsights/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&state=post</code>

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)

Example Call And Response

In this example, the API call is a request to return data for fixture UUID `85qb82cyhjboqkv50a13r8ktm` from the `/matchinsights` endpoint.

►XML example**Feed Output**

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

►<matchInsights> (XML only)

►<matchInfo>

►<description>

►<sport>

►<ruleset>

►<competition>

►<country>

►<tournamentCalendar>

►<stage>

►<series>

►<contestants> (XML only)

►<contestant>

►<country>

►<venue>

►<insights>

►<insight>

►<marketTypes>

►<marketType>

Betting Markets Dictionaries

Provides details about the Market id for betting insights

►Market ID and descriptions

►Market Grouping ID and descriptions

►Market Metric ID and descriptions